

MEAN CHAT: APLICACIÓN DE MENSAJERÍA EN TIEMPO REAL

Jonás Martínez Cuesta

PROYECTO DE DAW | IES Bernaldo de Quirós

Curso académico: 2025/2026

Mayo 2026

Contenido

Resumen	3
Palabras clave	3
Introducción	4
Objetivos	5
Análisis del contexto y estado del arte	6
Análisis del contexto	6
Estado del arte	6
Innovación.....	7
DAFO	7
Planificación	8
Temporalización del proyecto	8
Recursos humanos y materiales	9
Presupuesto y financiación	9
Análisis del sistema	10
Definición de requisitos	10
Requisitos funcionales:	10
Requisitos no funcionales:	10
Identificación de los subsistemas	11
Análisis de casos de uso	11
Análisis de interfaces	12
Especificación del plan de pruebas.....	12
Diseño del sistema	13
Arquitectura del sistema.....	13
Diseño de clases.....	13
Diseño de la base de datos	14
Otros diagramas.....	14
Diseño de interfaces	14
Implementación	16
Lenguajes de programación.....	16
Herramientas utilizadas	17
Problemas encontrados	17

Desarrollo de las pruebas	18
Pruebas de código.....	18
Pruebas de usabilidad.....	18
Pruebas de accesibilidad.....	18
Manuales del sistema	19
Manual de despliegue.....	19
Manual de usuario	19
Conclusiones	20
Desarrollo del proyecto	20
Futuras ampliaciones	20
Bibliografía	21
ANEXOS	22

Resumen

MEAN Chat es una aplicación de mensajería en tiempo real desarrollada con la pila MEAN (MongoDB, Express, Angular, Node.js) e integración con Socket.io. La aplicación permite a los usuarios registrarse, iniciar sesión y participar en conversaciones privadas y grupales con mensajería instantánea. Incorpora autenticación basada en tokens JWT, comunicación bidireccional mediante WebSockets, paginación de mensajes, compartición de ubicaciones geográficas y soporte para emojis. El sistema se compone de un backend REST con eventos Socket.io y un frontend Angular 21 con Ionic para una interfaz responsive y moderna. El despliegue se realiza mediante Docker Compose, integrando los servicios de base de datos, backend y frontend en contenedores independientes y reproducibles.

Palabras clave

MEAN stack, Angular, Node.js, Express, MongoDB, Socket.io, WebSockets, JWT, tiempo real, chat, Ionic, Docker, TypeScript

Introducción

La comunicación instantánea se ha convertido en una necesidad cotidiana tanto en contextos personales como profesionales. Aplicaciones como WhatsApp, Telegram o Discord han transformado la forma en que las personas interactúan, y han generado una demanda creciente de plataformas de mensajería modernas, escalables y seguras.

Este proyecto surge de la observación de que desarrollar una aplicación de mensajería en tiempo real constituye un reto técnico completo que abarca todos los aspectos del desarrollo web: diseño de base de datos, API REST, comunicación por WebSockets, autenticación segura y construcción de una interfaz de usuario reactiva. Representa un escenario idóneo para poner en práctica los conocimientos adquiridos durante el ciclo formativo de DAW.

MEAN Chat implementa todas las funcionalidades esenciales de una aplicación de mensajería moderna: registro e inicio de sesión con contraseña hasheada mediante bcrypt, creación de chats privados y grupales, envío y recepción de mensajes en tiempo real, compartición de ubicación geográfica y selección de emojis. La arquitectura adoptada —separación total entre frontend y backend, contenedores Docker y comunicación vía API REST y WebSockets— garantiza la escalabilidad y mantenibilidad del sistema.

Objetivos

Objetivo general: Desarrollar una aplicación web de mensajería en tiempo real full-stack usando la pila MEAN (MongoDB, Express, Angular, Node.js) con comunicación por WebSockets.

Objetivos específicos:

1. Implementar un sistema de autenticación seguro basado en JWT con registro, inicio de sesión y cierre de sesión.
2. Desarrollar una API REST con Express 5 para la gestión de usuarios y chats, documentada con Swagger UI.
3. Implementar comunicación en tiempo real mediante Socket.io para el envío y recepción de mensajes instantáneos.
4. Diseñar e implementar un modelo de datos NoSQL en MongoDB para usuarios, chats y mensajes utilizando Mongoose como ODM.
5. Construir una interfaz de usuario responsive y moderna con Angular 21 e Ionic.
6. Incorporar funcionalidades avanzadas: compartición de ubicación geográfica en tiempo real y selector de emojis.
7. Containerizar la aplicación completa con Docker Compose para facilitar el despliegue en cualquier entorno.
8. Implementar guards de autenticación y un interceptor HTTP para la gestión automática del token JWT en el frontend.

Análisis del contexto y estado del arte

Este apartado analiza el mercado de las aplicaciones de mensajería en tiempo real, las alternativas existentes, las innovaciones que aporta MEAN Chat y un análisis DAFO del proyecto.

Análisis del contexto

El mercado de la mensajería instantánea es uno de los segmentos más activos de la industria tecnológica. WhatsApp supera los 2.000 millones de usuarios activos, Telegram los 900 millones y Discord los 150 millones. En el ámbito empresarial, Slack y Microsoft Teams dominan la comunicación interna con millones de organizaciones como clientes.

El nicho de mercado de MEAN Chat son usuarios técnicos y pequeñas organizaciones que necesitan una plataforma de chat autodesplegable, open-source y personalizable, donde tengan control total sobre sus datos, sin depender de servidores de terceros. El nicho se caracteriza por una creciente preocupación por la privacidad y la soberanía digital, y por la preferencia de soluciones con stack JavaScript por su ecosistema unificado.

Estado del arte

Las principales alternativas en el mercado son:

WhatsApp / Telegram: Plataformas masivas de mensajería móvil. Código cerrado, dependientes de infraestructura propia. No autodesplegables.

Discord: Orientada a comunidades. Soporte de canales de texto, voz y vídeo. Código cerrado, no autodesplegable.

Slack: Enfocada en equipos empresariales. Modelo freemium con limitaciones en el plan gratuito. Código cerrado.

Rocket.Chat: Chat web open-source y autodesplegable con stack JavaScript. Mayor complejidad de despliegue y configuración.

Matrix / Element: Protocolo federado y open-source. Alta privacidad, pero complejo de desplegar y administrar.

Innovación

MEAN Chat aporta las siguientes innovaciones respecto a las alternativas existentes:

- Stack JavaScript unificado (TypeScript en frontend, JavaScript en backend), lo que simplifica el desarrollo y mantenimiento.
- Arquitectura moderna con Angular 21 e Ionic, que permite compatibilidad web y móvil desde el mismo código base.
- Compartición de ubicación geográfica en tiempo real como tipo de mensaje nativo, usando la API de Geolocalización del navegador y mapas estáticos de Geoapify.
- Despliegue completo con Docker Compose en un solo comando, integrando MongoDB, backend y frontend.
- API REST completamente documentada con Swagger UI, accesible en /api-docs.
- Rate limiting integrado para proteger los endpoints de autenticación frente a ataques de fuerza bruta.

DAFO

Fortalezas:

- Stack JavaScript unificado (TypeScript/JavaScript) en todo el proyecto.
- Arquitectura moderna con Angular 21, Ionic y Express 5.
- Autenticación JWT segura con bcrypt (10 salt rounds).
- Comunicación en tiempo real con Socket.io 4.
- Contenedores Docker para despliegue reproducible.
- API documentada con Swagger UI.
- Rate limiting para proteger los endpoints de autenticación.

Debilidades:

- Aplicación en fase de desarrollo, sin despliegue en producción real.
- Sin soporte de videollamadas o llamadas de audio.
- Sin cifrado end-to-end de mensajes.
- Cobertura de pruebas unitarias limitada.

Oportunidades:

- Creciente demanda de plataformas autodesplegables motivada por la privacidad y la soberanía de datos.
- El mercado de comunicación en equipos técnicos sigue en expansión.
- Posibilidad de añadir canales de voz/vídeo con WebRTC.
- Expansión a aplicación móvil nativa con Capacitor (iOS y Android).

Amenazas:

- Competencia de soluciones maduras como Rocket.Chat o Mattermost.
- Cambios en las APIs de terceros (Geoapify, EmojiAPI).
- Evolución rápida del ecosistema JavaScript (breaking changes en Angular, Ionic).

Planificación

El proyecto se ha planificado y ejecutado a lo largo del curso académico 2025/2026. La duración total fue de aproximadamente 8 meses, desde octubre de 2025 hasta mayo de 2026.

Temporalización del proyecto

El desarrollo se dividió en las siguientes fases:

- Octubre – Noviembre 2025: Análisis y diseño del sistema. Definición de requisitos, diseño de la base de datos y arquitectura del sistema.
- Noviembre – Diciembre 2025: Desarrollo del backend. Modelos Mongoose, API REST con Express, autenticación JWT y eventos Socket.io.
- Enero – Marzo 2026: Desarrollo del frontend. Módulo de autenticación (login/signup), listado de chats y vista de conversación.
- Marzo – Abril 2026: Funcionalidades avanzadas. Compartición de ubicación, selector de emojis y notificaciones en tiempo real.
- Abril – Mayo 2026: Integración frontend-backend. Pruebas del flujo completo y corrección de errores.
- Mayo 2026: Containerización con Docker, pruebas finales, documentación y redacción de la memoria.

Recursos humanos y materiales

Recursos humanos:

- 1 desarrollador: Jonás Martínez Cuesta (rol: desarrollador full-stack).

Recursos materiales:

- Hardware: Equipo de desarrollo personal con Windows 11.
- Software: VS Code, Docker Desktop, Node.js 22, npm 11.8.0, Git.
- Servicios externos: Geoapify API (mapas estáticos), EmojiAPI (emojis), GitHub (repositorio y CI/CD).
- Base de datos: MongoDB 7 en contenedor Docker.

Presupuesto y financiación

El proyecto se ha desarrollado exclusivamente con herramientas open-source o en su plan gratuito:

- VS Code, Node.js, MongoDB, Docker: 0 € (open-source / gratuito).
- GitHub: 0 € (plan gratuito).
- Geoapify API: 0 € (plan gratuito: hasta 3.000 peticiones/día).
- EmojiAPI: 0 € (plan gratuito).
- Coste total de desarrollo: 0 €.
- Coste estimado de despliegue en producción (VPS 2 vCPU, 4 GB RAM): ~10 €/mes.

Análisis del sistema

Definición de requisitos

Requisitos funcionales:

- 1) RF-01: El sistema permitirá a los usuarios registrarse con username, nombre, apellido, email y contraseña.
- 2) RF-02: El sistema permitirá iniciar sesión con username y contraseña, emitiendo un token JWT de 1 hora de duración.
- 3) RF-03: Los usuarios autenticados podrán crear chats privados (2 participantes) o grupales (N participantes).
- 4) RF-04: Los usuarios podrán ver la lista de sus chats ordenada por última actividad.
- 5) RF-05: Los usuarios podrán enviar y recibir mensajes de texto en tiempo real dentro de un chat.
- 6) RF-06: Los usuarios podrán enviar su ubicación geográfica como tipo de mensaje especial.
- 7) RF-07: Los usuarios podrán seleccionar emojis para incluir en sus mensajes de texto.
- 8) RF-08: Los mensajes de un chat se cargarán de forma paginada (30 mensajes por página).
- 9) RF-09: El sistema mostrará el último mensaje de cada chat en la lista, actualizado en tiempo real.
- 10) RF-10: Los usuarios podrán cerrar sesión, eliminando el token local y desconectando el WebSocket.

Requisitos no funcionales:

- 1) RNF-01: Las contraseñas se almacenarán hasheadas con bcrypt (10 salt rounds). Nunca en texto plano.
- 2) RNF-02: Todos los endpoints protegidos requerirán autenticación mediante token JWT Bearer.
- 3) RNF-03: El sistema limitará las peticiones a los endpoints de autenticación a 20 por 15 minutos por IP.
- 4) RNF-04: La aplicación completa se desplegará en contenedores Docker gestionados con Docker Compose.
- 5) RNF-05: La API REST estará documentada con Swagger UI accesible en la ruta /api-docs.

Identificación de los subsistemas

El sistema se divide en los siguientes subsistemas y módulos:

- Subsistema de Autenticación (Backend): Registro, login, generación de tokens JWT, middleware de autenticación HTTP y validación en Socket.io.
- Subsistema de Chats (Backend): Creación y listado de chats privados/grupales, gestión de miembros y notificación en tiempo real via Socket.io.
- Subsistema de Mensajería en tiempo real (Backend): Eventos Socket.io: join-chat, leave-chat, chat-message, chat-updated. Persistencia en MongoDB.
- Subsistema de Base de Datos: Modelos Mongoose (User, Chat, Message), índices y relaciones entre colecciones.
- Módulo de Autenticación (Frontend): Componentes Login y Signup, guards de navegación (AuthGuard, HomeGuard), interceptor HTTP para el token JWT.
- Módulo de Chat (Frontend): Componentes ChatHome, ChatList, ChatView, ChatInput, ChatHeader, ChatWidget, AccountPanel, CreateChat, ChatListActions.
- Módulo de Mensajes (Frontend): Componentes MessageList, MessageItem y MessageContent para renderizar el historial de mensajes.
- Servicios compartidos (Frontend): SocketService, AuthService, ChatService, MessageService, CurrentUserService, UserService, NotificationService.

Análisis de casos de uso

CU-01: Registro de usuario

Actor: Usuario no autenticado.

Flujo: El usuario accede a /auth/signup → rellena el formulario (username, nombre, apellido, email, contraseña) → el sistema valida los datos → hashea la contraseña con bcrypt → crea el usuario en la base de datos → devuelve confirmación al frontend.

CU-02: Inicio de sesión

Actor: Usuario no autenticado.

Flujo: El usuario introduce credenciales en /auth/login → el sistema verifica contra la base de datos → genera un JWT con duración de 1 hora → el frontend almacena el token y conecta el WebSocket con el token como autenticación.

CU-03: Crear chat

Actor: Usuario autenticado.

Flujo: El usuario accede a /home/create-chat → selecciona miembros y asigna un nombre → elige participantes → el sistema crea el chat en la BD → notifica a todos los miembros via Socket.io (evento chat-created).

CU-04: Enviar mensaje

Actor: Usuario autenticado en un chat.

Flujo: El usuario escribe y envía un mensaje → el frontend emite el evento "chat-message" via Socket.io → el backend persiste el mensaje en MongoDB → actualiza el lastMessage del chat → emite el mensaje a todos los sockets de la sala → el frontend lo muestra en tiempo real.

CU-05: Ver historial de mensajes

Actor: Usuario autenticado.

Flujo: El usuario abre un chat → el frontend solicita GET /chat/:id/messages?page=1 → el backend verifica que el usuario es miembro → devuelve los mensajes paginados (30/página) → el frontend renderiza el historial con scroll.

Análisis de interfaces

La interfaz sigue una estructura de dos paneles en pantallas de escritorio:

- Panel izquierdo: Lista de chats con preview del último mensaje (ChatList) y panel de cuenta del usuario con nombre y opción de cierre de sesión (AccountPanel).
- Panel principal (derecho): Vista del chat activo con cabecera (ChatHeader), lista de mensajes con scroll (MessageList) y campo de entrada con selector de emojis y botón de ubicación (ChatInput). Si no hay chat activo, se muestra el formulario de creación de chat (CreateChat).

En dispositivos móviles, la interfaz adapta la navegación para mostrar un panel a la vez, aprovechando las capacidades responsive de Ionic.

Especificación del plan de pruebas

Se planificaron los siguientes tipos de pruebas:

- Pruebas unitarias de backend con Jest 30: cobertura de modelos Mongoose, rutas REST y middleware de autenticación.
- Pruebas unitarias de frontend con Vitest 4: cobertura de servicios Angular (AuthService, ChatService, MessageService) y componentes principales.
- Pruebas de integración: verificación del flujo completo desde el registro hasta el envío de mensajes en tiempo real.

- Pruebas de usabilidad: navegación en escritorio, tablet y móvil con usuarios reales.
- Pruebas de accesibilidad: validación con Lighthouse (Chrome DevTools) para contraste, ARIA y navegación por teclado.

Diseño del sistema

Arquitectura del sistema

La aplicación sigue una arquitectura cliente-servidor de tres capas, desplegada en contenedores Docker:

- Capa de presentación: Angular 21 + Ionic (SPA) servida por Nginx en el puerto 80. Comunicación con el backend vía HTTP (REST API) y WebSocket (Socket.io).
- Capa de lógica de negocio: Node.js + Express 5 con Socket.io en el puerto 3000. Gestiona autenticación, lógica de chats y mensajería en tiempo real.
- Capa de datos: MongoDB 7 en contenedor Docker con volumen persistente (mongo_data). Almacena colecciones users, chats y messages.

Los tres servicios (mongo, backend, frontend) se comunican a través de una red Docker interna (chat-net), con acceso externo únicamente a los puertos 80 (frontend) y 3000 (backend).

Diseño de clases

Modelos de datos (Mongoose – Backend):

- User: { username: String (único), passwordHash: String, name: String, surname: String, email: String (único) }. Métodos estáticos: hashPassword(). Métodos de instancia: isValidPassword().
- Chat: { isGroup: Boolean, members: [ObjectId → User] (mín. 2), name: String, lastMessage: String, createdAt: Date, updatedAt: Date }. Índice en el campo members.
- Message: { chat: ObjectId → Chat, sender: ObjectId → User, content: Mixed, type: String (enum: "text" | "location"), createdAt: Date }.

Interfaces TypeScript (Frontend):

- User: { _id, username, name, surname, email }
- Chat: { _id, name, isGroup, members: User[], lastMessage, createdAt, updatedAt }
- Message: { _id, chat, sender: User, content, type, createdAt }
-

Diseño de la base de datos

La base de datos MongoDB (mean-chat) utiliza tres colecciones con las siguientes relaciones:

- users: Documentos de usuario. Campos clave: username (índice único), email (índice único), passwordHash.
- chats: Documentos de chat. El campo members es un array de referencias a documentos de users (ObjectId). Tiene un índice en members para búsquedas eficientes por usuario.
- messages: Documentos de mensaje. Referencia al chat (chat: ObjectId) y al emisor (sender: ObjectId). El campo content es de tipo Mixed para soportar texto y objetos de ubicación geográfica.

Relaciones: Un usuario participa en múltiples chats. Un chat contiene múltiples mensajes. Un mensaje pertenece a un único chat y fue enviado por un único usuario.

Otros diagramas

El repositorio incluye un diagrama ER en formato Mermaid en el archivo db-model.md que muestra las entidades USER, CHAT y MESSAGE con sus atributos y relaciones. Se pueden incluir además diagramas de secuencia para los flujos de autenticación y mensajería en tiempo real.

Diseño de interfaces

La aplicación presenta un diseño oscuro (dark mode) con un sistema de diseño basado en Ionic y variables CSS personalizadas. La paleta de colores utiliza tonos púrpura/índigo como color primario. La fuente utilizada es Inter (incluida como asset local).

Pantallas principales implementadas:

- `/auth/login`: Formulario de inicio de sesión con campos username y password.
- `/auth/signup`: Formulario de registro con username, nombre, apellidos, email y contraseña (con confirmación). Incluye validación de coincidencia de contraseñas.
- `/home`: Vista principal. Panel lateral con AccountPanel + ChatList. Área principal con router-outlet para las vistas de chat.
- `/home/:chatId`: Vista de conversación con ChatHeader (nombre del chat), MessageList (historial paginado con scroll infinito) y ChatInput (texto + emojis + ubicación).
- `/home/create-chat`: Formulario para crear nuevos chats con selección de miembros y tipo (privado/grupo).

Implementación

Lenguajes de programación

- JavaScript (ES2022): Backend Node.js — servidor Express, modelos, rutas y eventos Socket.io.
- TypeScript 5.9: Frontend Angular — servicios, componentes, guards, interceptores y modelos de datos.
- HTML5: Plantillas de componentes Angular.
- SCSS: Estilos de componentes y estilos globales de la aplicación.
- YAML: Configuración de Docker Compose y definición de la API con Swagger.

Frameworks y librerías – Backend:

- Express 5.1.0: Framework web para Node.js. Enrutamiento, middleware y API REST.
- Socket.io 4.8.1: Comunicación bidireccional en tiempo real via WebSockets.
- Mongoose 8.18.3: ODM para MongoDB. Esquemas, validaciones y consultas.
- jsonwebtoken 9.0.2: Generación y verificación de tokens JWT.
- bcrypt 6.0.0: Hashing seguro de contraseñas con salt aleatorio (10 rounds).
- express-rate-limit 8.5.2: Protección de endpoints frente a ataques de fuerza bruta.
- swagger-ui-express 5.0.1: Documentación interactiva de la API REST.
- dotenv 17.2.3: Gestión de variables de entorno desde archivos .env.

Frameworks y librerías – Frontend:

- Angular 21.1.0: Framework SPA. Componentes standalone, servicios, routing, guards e interceptores HTTP.
- Ionic 8.7.17: Librería de componentes UI mobile-first y responsive.
- socket.io-client 4.8.3: Cliente WebSocket para la comunicación en tiempo real desde Angular.
- RxJS 7.8: Programación reactiva. Observables y BehaviorSubjects para streams de datos.
- Capacitor 8.3.3: Acceso a APIs nativas del dispositivo (geolocalización GPS).

Herramientas utilizadas

- VS Code: Editor de código principal con extensiones para TypeScript, Angular, Docker y ESLint.
- Docker Desktop: Gestión de contenedores en desarrollo y para despliegue.
- Git / GitHub: Control de versiones y repositorio remoto (github.com/Yonoveleta/mean-chat).
- MongoDB Compass: GUI para inspección y administración de la base de datos MongoDB.
- Swagger UI: Documentación y prueba interactiva de la API REST (accesible en [/api-docs](#)).
- Draw.io: Creación de diagramas de arquitectura y modelo de datos.
- Geoapify API: Generación de mapas estáticos para los mensajes de ubicación.
- EmojiAPI: Proveedor de datos de emojis para el selector de expresiones.
- Jest 30: Framework de testing para el backend Node.js con cobertura de código.
- Vitest 4.0: Framework de testing para el frontend Angular integrado con Angular CLI.
- GitHub Actions: Integración continua (CI/CD) con workflow de tests automatizados.
- Nodemon 3.1.10: Reinicio automático del servidor backend en modo desarrollo.
- Prettier: Formateo automático de código TypeScript, JavaScript y HTML.

Problemas encontrados

Durante el desarrollo se encontraron los siguientes problemas y sus soluciones:

- CORS en Docker: Las peticiones del frontend al backend fallaban por políticas CORS al ejecutar en contenedores. Solución: configurar la variable de entorno `FRONTEND_URL` en el backend y establecer correctamente el origen permitido en Express y Socket.io.
- Resolución DNS en Docker: Las conexiones del backend a MongoDB fallaban porque Node.js resolvía IPv6 antes que IPv4. Solución: añadir `require("dns").setDefaultResultOrder("ipv4first")` al inicio de `server.js`.
- Sincronización de eventos Socket.io: Los mensajes podían perderse si el join-chat no se había completado. Solución: el frontend espera confirmación del join antes de mostrar la UI del chat.
- Compatibilidad Angular 21 + Ionic: La versión 21 de Angular introdujo cambios en el sistema de compilación. Solución: actualizar la configuración del builder de Ionic en `angular.json`.

- Gestión automática del JWT en el frontend: Asegurar que el token se adjunta a todas las peticiones. Solución: implementar un interceptor HTTP (AuthInterceptor) que añade el header "Authorization: Bearer <token>" automáticamente.

Desarrollo de las pruebas

Pruebas de código

El backend utiliza Jest 30 como framework de pruebas unitarias. La configuración en package.json incluye generación de cobertura de código y reportes en formato JUnit (jest-junit) para integración con GitHub Actions CI/CD. Los tests se ejecutan con npm test. Se han implementado tests para los modelos, rutas REST y middleware de autenticación.

El frontend utiliza Vitest 4.0 integrado con Angular CLI (ng test). Se han implementado tests unitarios para los servicios principales (AuthService, ChatService, MessageService, CurrentUserService), los guards de navegación (AuthGuard, HomeGuard) y los componentes principales (LoginComponent, SignupComponent, ChatListComponent, ChatHomeComponent).

Pruebas de usabilidad

Se realizaron pruebas con usuarios reales en tres tipos de dispositivo: escritorio (1920×1080), tablet (768×1024) y móvil (375×812). Se verificaron los flujos principales: registro, inicio de sesión, creación de chat, envío de mensajes de texto, envío de ubicación, selección de emojis, navegación entre conversaciones y cierre de sesión.

Los usuarios encontraron la interfaz intuitiva para las funciones principales. Se detectó que en dispositivos móviles la transición entre el panel de lista de chats y la vista de conversación podría mejorarse con gestos de deslizamiento nativos.

Pruebas de accesibilidad

Se utilizó Lighthouse en Chrome DevTools para validar la accesibilidad de las páginas principales (login, signup y vista de chat). Se verificaron: contraste de colores suficiente para textos sobre fondo oscuro, etiquetas ARIA en elementos interactivos, navegación por teclado funcional y atributos alt en imágenes e iconos.

Manuales del sistema

Manual de despliegue

Requisitos previos:

- Docker Desktop instalado y en ejecución (versión 24 o superior).
- Git instalado.
- Puertos 80 y 3000 disponibles en la máquina host.

Pasos de despliegue:

1. Clonar el repositorio: `git clone --recurse-submodules https://github.com/RealYonoveleta/mean-chat`
2. Acceder al directorio del proyecto: `cd mean-chat`
3. Construir e iniciar todos los contenedores: `docker compose up --build`
4. Esperar a que todos los servicios arranquen (aprox. 1–2 minutos).
5. Acceder a la aplicación en el navegador: `http://localhost`
6. La API REST estará disponible en: `http://localhost:3000`

Comandos de gestión:

- Detener los contenedores (conservando datos): `docker compose down`
- Detener y eliminar todos los datos: `docker compose down -v`
- Ver logs de un servicio: `docker compose logs backend`

Manual de usuario

Guía de uso de las funciones principales:

1. Registro: Accede a `/auth/signup` y rellena el formulario con nombre, apellido, username, email y contraseña (repítela para confirmar). Pulsa "Registrarse".
2. Inicio de sesión: Introduce tu username y contraseña en `/auth/login`. Pulsa "Iniciar sesión".
3. Crear un chat: Pulsa el botón "+" en la lista de chats. Selecciona los participantes, asigna un nombre y elige si es chat privado o grupo. Confirma la creación.
4. Enviar mensajes: Selecciona un chat de la lista. Escribe en el campo de texto inferior y pulsa Enter o el botón de envío.
5. Compartir ubicación: Pulsa el icono de pin/ubicación en la barra de entrada. El navegador solicitará permiso de geolocalización. Una vez concedido, se enviará tu posición como mensaje con mapa estático.
6. Selector de emojis: Pulsa el icono de emoji para abrir el selector de expresiones y añadir emojis al mensaje.
7. Cerrar sesión: Accede al panel de cuenta en el panel lateral y pulsa el botón de cerrar sesión.

Conclusiones

Desarrollo del proyecto

El desarrollo de MEAN Chat ha supuesto un reto técnico enriquecedor que ha permitido integrar múltiples tecnologías modernas en un proyecto cohesionado y funcional. El proyecto cubre el ciclo completo del desarrollo de una aplicación web: desde el diseño de la base de datos y la API REST hasta la interfaz de usuario y el despliegue con Docker.

El mayor desafío técnico fue la coordinación entre la capa HTTP y la capa WebSocket, especialmente en la gestión del estado de la conexión durante el login/logout y en la sincronización de eventos en tiempo real (mensajes, actualizaciones de chats) entre múltiples clientes simultáneos.

El resultado es una aplicación funcional con características comparables a aplicaciones profesionales: autenticación segura con JWT y bcrypt, mensajería en tiempo real, soporte de chats grupales y privados, compartición de ubicación y una interfaz moderna y responsive. La containerización con Docker garantiza que el proyecto sea reproducible y desplegable en cualquier entorno. Creo que el proyecto tiene potencial para evolucionar a una herramienta real con las mejoras indicadas en el apartado de futuras ampliaciones.

Futuras ampliaciones

- Cifrado end-to-end de mensajes para mayor privacidad.
- Soporte para envío de archivos adjuntos (imágenes, documentos, audio).
- Canales de audio y videollamada con WebRTC.
- Sistema de roles y permisos en chats grupales (admin, moderador, miembro).
- Notificaciones push para mensajes recibidos en segundo plano.
- Aplicación móvil nativa con Capacitor (iOS y Android).
- Modo de edición y eliminación de mensajes.
- Indicador de "escribiendo..." en tiempo real.
- Autenticación con OAuth2 (Google, GitHub).
- Mensajes con expiración automática (modo efímero).

Bibliografía

- Documentación oficial de Angular 21: <https://angular.dev>
- Documentación oficial de Ionic Framework: <https://ionicframework.com/docs>
- Documentación oficial de Socket.io 4: <https://socket.io/docs/v4>
- Documentación oficial de Express 5: <https://expressjs.com>
- Documentación oficial de Mongoose 8: <https://mongoosejs.com/docs>
- Documentación oficial de MongoDB: <https://www.mongodb.com/docs>
- Documentación oficial de Docker: <https://docs.docker.com>
- JSON Web Tokens (JWT) – RFC 7519: <https://tools.ietf.org/html/rfc7519>
- OWASP Top 10 – 2021: <https://owasp.org/Top10>
- MDN Web Docs – Geolocation API: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API
- RxJS Documentation: <https://rxjs.dev/guide/overview>
- bcrypt npm package: <https://www.npmjs.com/package/bcrypt>
- Geoapify Static Maps API: <https://www.geoapify.com/static-maps-api>
- Capacitor Geolocation Plugin: <https://capacitorjs.com/docs/apis/geolocation>

ANEXOS

Anexo A: Diagrama ER de la base de datos

El diagrama ER completo en formato Mermaid se encuentra en el archivo db-model.md del repositorio. Las entidades son USER, CHAT y MESSAGE con las siguientes relaciones:

```
USER ||--o{ MESSAGE : sends (un usuario envía múltiples mensajes)
USER ||--o{ CHAT : participates_in (un usuario participa en múltiples chats)
CHAT ||--o{ MESSAGE : contains (un chat contiene múltiples mensajes)
```

Anexo B: Documentación de la API REST

La documentación completa de todos los endpoints de la API está disponible en Swagger UI. Una vez desplegada la aplicación, acceder a: <http://localhost:3000/api-docs>

Resumen de endpoints documentados:

- POST /auth/register — Registro de nuevo usuario
- POST /auth/login — Inicio de sesión y obtención de token JWT
- GET /auth/me — Obtener perfil del usuario autenticado (requiere JWT)
- GET /auth/users — Listar todos los usuarios excepto el actual (requiere JWT)
- POST /chat — Crear un nuevo chat (requiere JWT)
- GET /chat — Obtener todos los chats del usuario (requiere JWT)
- GET /chat/:id — Obtener un chat por ID (requiere JWT)
- GET /chat/:id/messages?page=1&limit=30 — Obtener mensajes paginados (requiere JWT)

Anexo C: Repositorio del proyecto

GitHub: <https://github.com/RealYonoveleta/mean-chat>

El repositorio incluye el código fuente completo, el Dockerfile de cada servicio, el docker-compose.yml y la documentación.